

ST445

Managing and Visualising Data

Dr Leonardo Alves Dias



Agenda

- Introduction to ST445
- Introduction to Data
- Introduction to GitHub Workflows



THE LONDON SCHOOL
OF ECONOMICS AND
POLITICAL SCIENCE ■

INTRODUCTION TO ST445

Timetable

Lectures:

Tuesdays
11:00 – 13:00

Usually recorded

Dr Leonardo Alves Dias

Seminar 1

Thursday: 11:00 – 12:30 (*Not recorded*)

Dr Leonardo Alves Dias

Seminar 2

Thursday: 12:30 – 14:00 (*Not recorded*)

Dr Leonardo Alves Dias

Seminar 3

Thursday: 17:00 – 18:30 (*Not recorded*)

Dr Kaifang Zhou

Seminar 4

Friday: 13:30 – 15:00 (*Not recorded*)

Dr Kaifang Zhou

Resources, Assignments, and Support



Course: ST445 - Managing and Visualising Data 2026/27 | Moodle



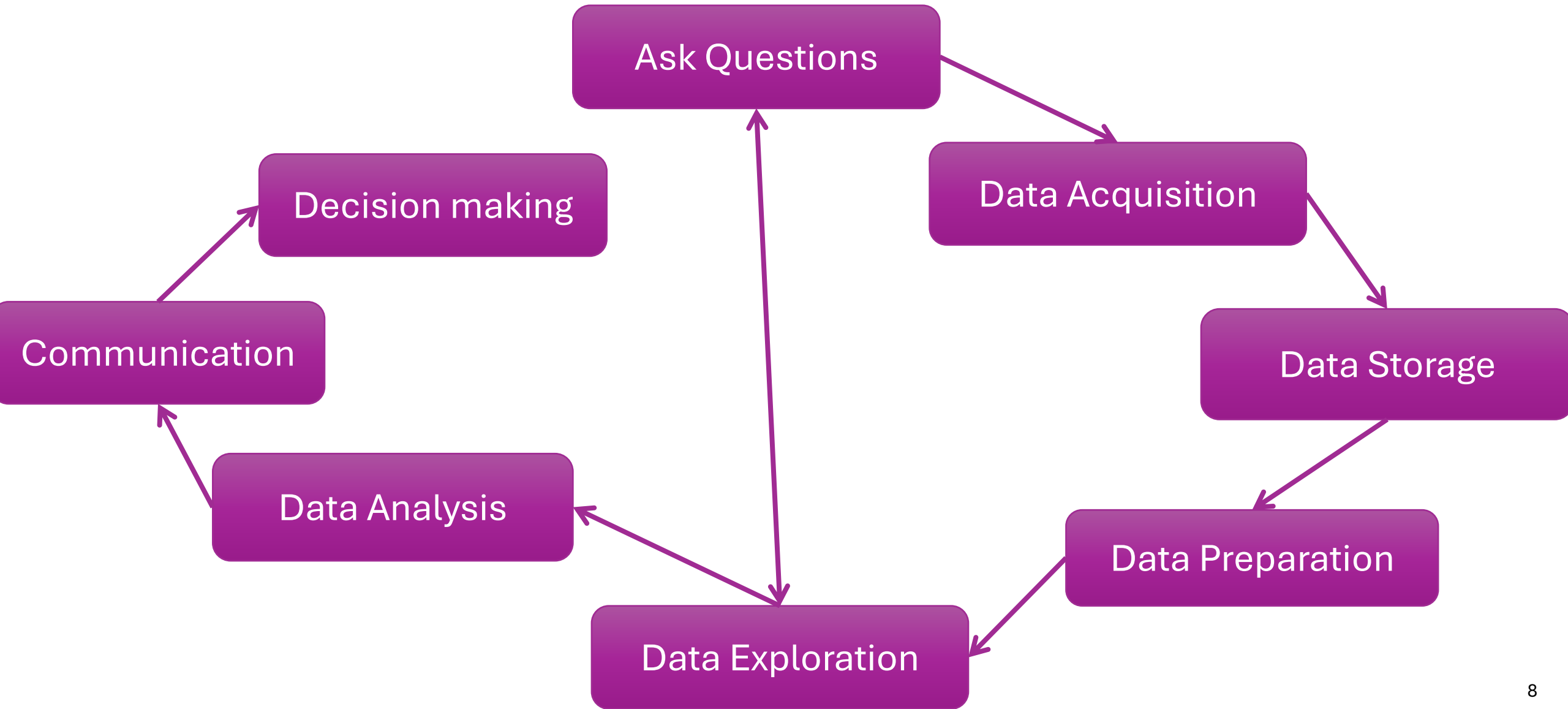
What is ST445 about?

- Principles and best practices for data manipulation and visualisation
- Provide hands-on experience in carrying out a full data science project cycle, from data acquisition to reporting with conclusions and visualisations using Python
- This course is NOT about state-of-the-art statistical modelling techniques/machine learning methods/artificial intelligence.
- Our department provides a range of other courses:
 - ST443 Machine Learning and Data Mining
 - ST447 Data Analysis and Statistical Methods
 - ST455 Reinforcement Learning
 - ST456 Deep Learning
 - ST457 Graph Data Analytics and Representation Learning
- Computer Programming course available (MY470)
- MY472 (Data for Data Scientists) has significant overlap with ST445

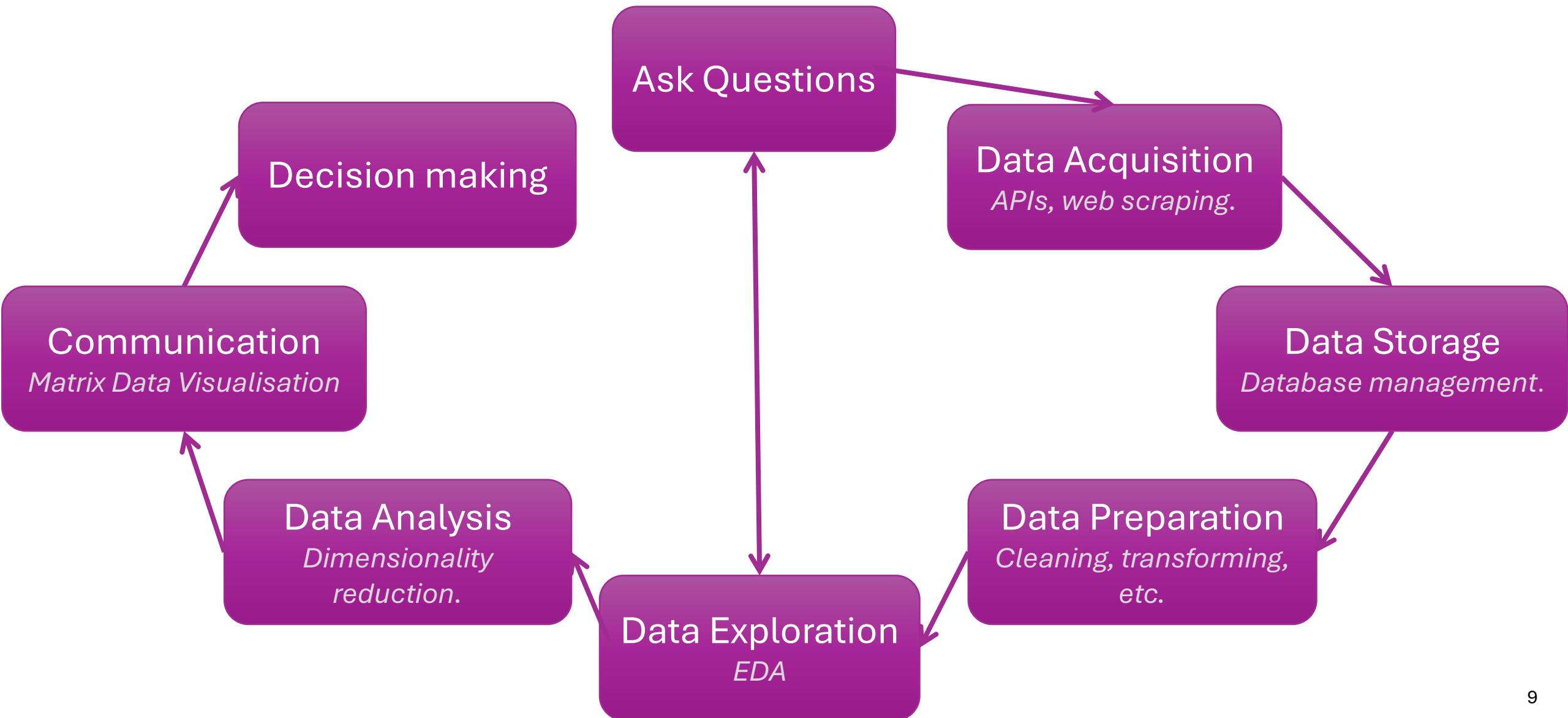
Who takes this course?

- Compulsory for MSc in Data Science and MSc in Health Data Science.
- Available for MPA in Data Science for Public Policy, MSc in Applied Social Data Science, MSc in Geographic Data Science, MSc in Statistics, MSc in Statistics (Financial Statistics), and others.

What is the data science project cycle?



What is the data science project cycle?



ST445 Topic schedule

Wk	Topic	Pipeline stage	LOs
1	Introduction to Data and Reproducible Workflows with GitHub	Foundations and version control	LO1, LO4, LO3
2	Collecting Data from the Web: Introduction to APIs and Web Scraping	Pipeline stage 1 — data collection	LO3, LO1
3	Representing Data: NumPy and pandas Data Structures	Pipeline stage 2 — representation	LO1, LO4
4	Data wrangling with pandas	Pipeline stage 3 — cleaning and preprocessing	LO4
5	Managing Data via Databases	Pipeline stage 4 — storage and query	LO2, LO3
6	Reading Week — no teaching	Consolidation	—
7	Exploratory Data Analysis and the Principles of Effective Visualisation	Pipeline stage 5 — exploration	LO5, LO6
8	Matrix Visualisation	Pipeline stage 5 — exploration continued	LO5, LO6
9	Visualising and Communicating Model Performance	Pipeline stage 5 — evaluation	LO5, LO6
10	Dimensionality Reduction for Visualisation	Pipeline stage 5 — high-dimensional exploration	LO5, LO6
11	Graph Data Visualisation and Communicating Data-Informed Decisions	Pipeline stage 6 — communication	LO6, LO5, LO2

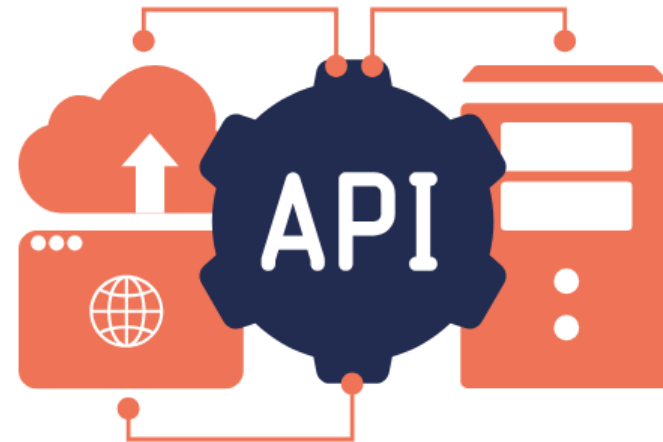
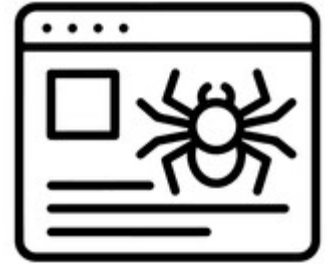
What will you get from the course?

1. Understand the main principles of data manipulation and visualisation.
2. Gain experience working with real-world data, tools, and techniques.
3. Capability to perform key steps in a data science project life cycle to address real-world problems .
4. Develop self-learning skills and critical thinking.

Data science tools used in the real world



colab



INTRODUCTION TO DATA

37.9

What is this?

The same value but different context

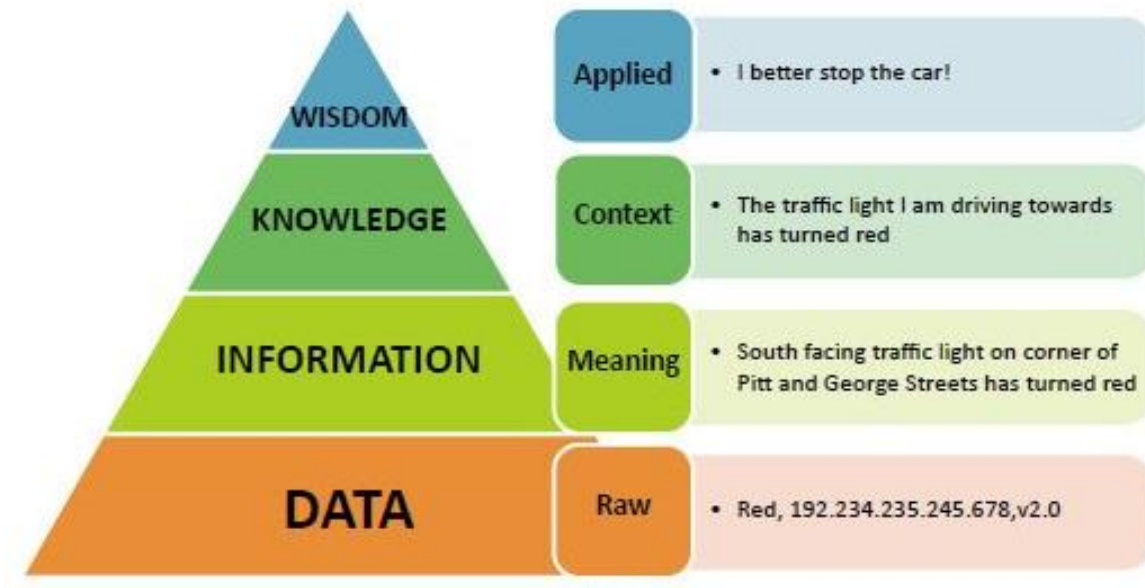
37.9

Unit	degrees Celsius
Measured	body temperature
Patient	4411
When	2026-10-05 08:15 UTC
Triage Category	2

Question it can now answer: Does this patient have a fever?

Data vs Information

- Data refers to raw, unorganised facts, numbers, symbols, or observations that have not yet been processed or given context ([Singh, N., 2024](#)).
- Meanwhile, information is when these facts and observations are analysed and presented in such a way that they are meaningful or can be used for decision-making.
- For example, a list of names is data. A register of who attended a class is information.



(Google, 2023)

Measurement

- Measurement is the systematic process of assigning numbers, symbols, or values to objects, traits, or events according to a set of standardised rules (Google, 2024).
- Four measurement scales:

Nominal

Data is split into distinct categories with no inherent order or ranking.

Properties: Classification only.

Examples: eye colour, gender, marital status, car brands.

Allowed Math: Frequencies, percentages, and mode.

Ordinal

Data is placed into categories that follow a meaningful order or ranking. The exact distance between values is unknown or unequal.

Properties: Classification + Order.

Examples: Likert scale (strongly agree to strongly disagree), finishing place in a race (1st, 2nd, 3rd), letter grades (A, B, C).

Allowed Math: Median, mode, frequencies.

Interval

Numerical data where the distance (interval) between adjacent values is equal and meaningful.

Properties: Classification + Order + Equal Intervals.

Examples: Temperature in Celsius or Fahrenheit, IQ scores, calendar years.

Allowed Math: Addition, subtraction, mean, median, standard deviation.

Ratio

The most precise numerical scale, combining equal intervals with an absolute, true zero point.

Properties: Classification + Order + Equal Intervals + True Zero (zero means the total absence of the variable).

Examples: Height, weight, age, temperature in Kelvin, financial income/salary.

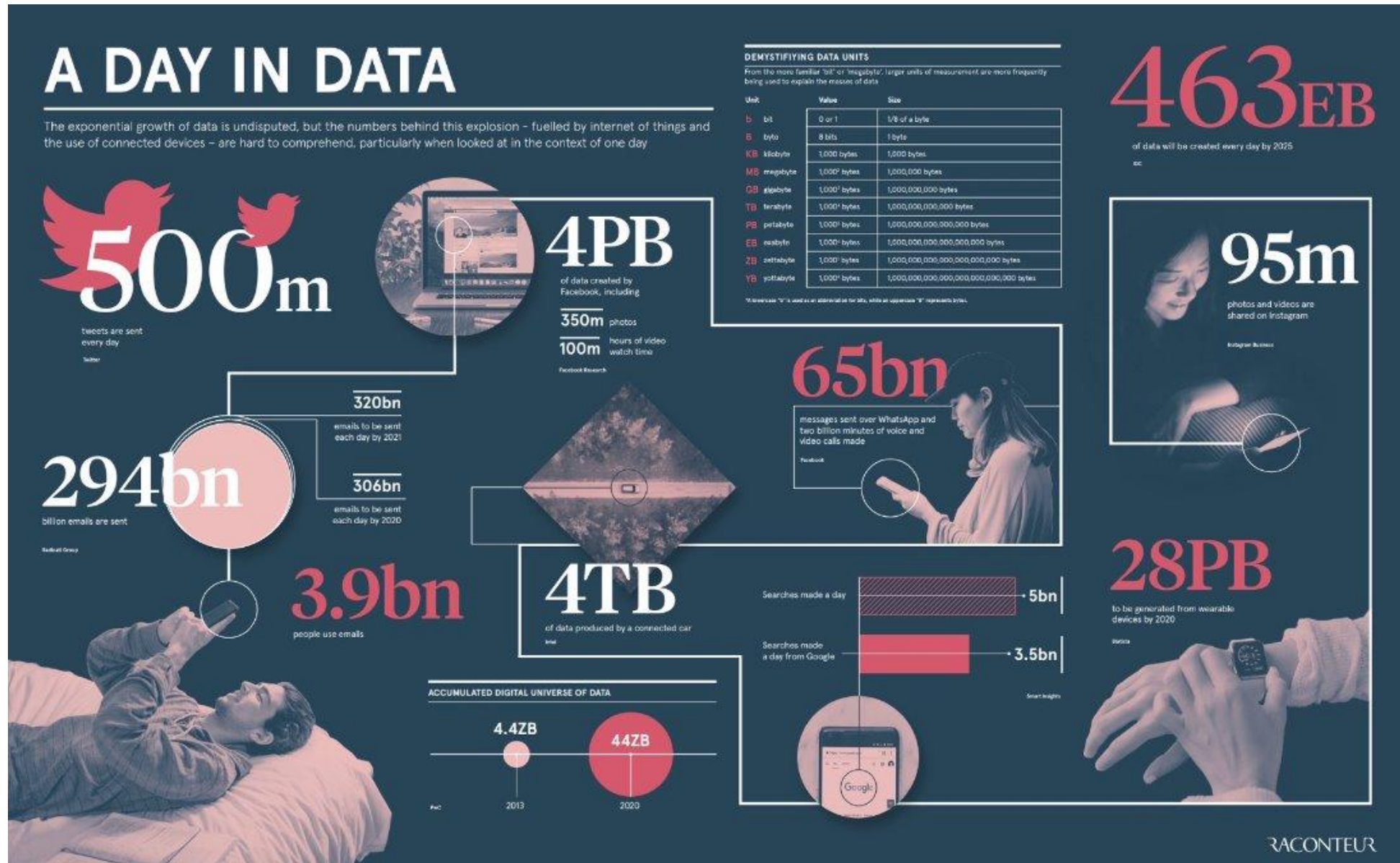
Allowed Math: All mathematical and statistical operations, including multiplication and division.

Data becomes more and more complicated ...

Changes in the world of data

- High-dimensional data ($p \gg n$), e.g., genetic data (dimension reduction, penalised regression, random projection)
- Functional data, e.g., time series, images (functional principal component analysis, deep learning)
- unstructured text data
- big data/massive data (subsampling, divide and conquer, parallel computing)
 - volume of data in the modern world: 90% of the world's data generated in the last two years ([Science Daily, 2013](#)).
 - and that was in 2013

Big Data Examples



STRUCTURE OF DATA

- Data can be structured in different schemas

Structured

Fixed schema. Every record has the same typed fields.

Examples: Database tables, clean CSV, spreadsheets with one header row.

Semi-structured

Self-describing keys or tags. Records can differ and nest.

Examples: JSON, XML, HTML, GeoJSON, email headers.

Unstructured

No schema a machine can use without interpretation.

Example: Free text, images, audio, scanned PDFs.

Unstructured data

The university has 5600 students.

John's ID is number 1, he is 18 years old and already holds a B.Sc. degree.

David's ID is number 2, he is 31 years old and holds a Ph.D. degree. Robert's ID is number 3, he is 51 years old and also holds the same degree as David, a Ph.D. degree.

Semi-structured data

```

<University>
  <Student ID="1">
    <Name>John</Name>
    <Age>18</Age>
    <Degree>B.Sc.</Degree>
  </Student>
  <Student ID="2">
    <Name>David</Name>
    <Age>31</Age>
    <Degree>Ph.D. </Degree>
  </Student>
  ....
</University>

```

Structured data

ID	Name	Age	Degree
1	John	18	B.Sc.
2	David	31	Ph.D.
3	Robert	51	Ph.D.
4	Rick	26	M.Sc.
5	Michael	19	B.Sc.

(Google, 2026)

More structure → less work before analysis

Where you can find each kind

Health records

Admissions table with coded diagnoses

Structured

Public spending

Departmental spend over £25,000, published monthly

Structured

Survey responses

Likert items plus a free-text comments field

Mixed

Free text

Parliamentary debates, product reviews, social posts

Unstructured

Images

Satellite tiles, scanned forms, X-rays

Unstructured

Geospatial files

GeoJSON boundaries, shapefiles

Semi-structured

Real datasets can be a mix of all three

resp_id	q1_satisfied	q2_recommend	comments	device
R0012	4	5	Delivery was late but support fixed it fast	{"os": "iOS", "app": "3.2"}
R0013	2	1	never again!!!	{"os": "Android"}
R0014	5	5		{"os": "web", "ref": "email"}

 Structured Unstructured text Semi-structured JSON inside a cell

- Structured data is often easier to manage and prepare for analysis. The more unstructured the data, the more preparation it will likely need.

Primitive data types

Type	Example	Python	What goes wrong
Integer	42	int	Leading zeros stripped from IDs: 00731 → 731
Floating point	37.9	float	Large IDs lose digits; rounding surprises
Boolean	True	bool	"Yes", "Y", "1", "TRUE" all meaning the same
String	"SEPTIN2"	str	Encoding damage; text read as dates or numbers
Date and time	2026-10-05T08:15Z	datetime	Day and month swapped; time zone dropped
Missing		None, NaN	Many spellings, and real values mistaken for missing

Missing data has many spellings

(blank)	NA	N/A	NULL	None
-999	.	?	0	unknown

The red ones are also real values.

Decide what missing values look like in each source, convert them explicitly when you load the data, and record your decision.

Standards remove the guessing

IEEE 754

Floating-point numbers

How is this number stored?

ISO 8601

Dates and times

Which moment is this?

Unicode and UTF-8

Text

Which characters are these bytes?

ISO 3166 and ISO 4217

Country and currency codes

Which country, which currency?

IEEE 754: floats are exact only so far

- The IEEE 754 is a technical standard for floating-point arithmetic to define how computers represent and calculate real numbers.

```
>>> 2**53
9007199254740992

>>> float(9007199254740993)
9007199254740992.0    # last digit lost
```

2^{53}

About 9×10^{15} . The largest integer a 64-bit float is guaranteed to hold exactly.

ISO 8601: one way to write a moment

ISO 8601 is the international standard for representing dates and times in a clear, unambiguous, and machine-readable way.

05/10/2026

5 October in the UK. 10 May in the US.

2026-10-05T14:30Z

date, year first

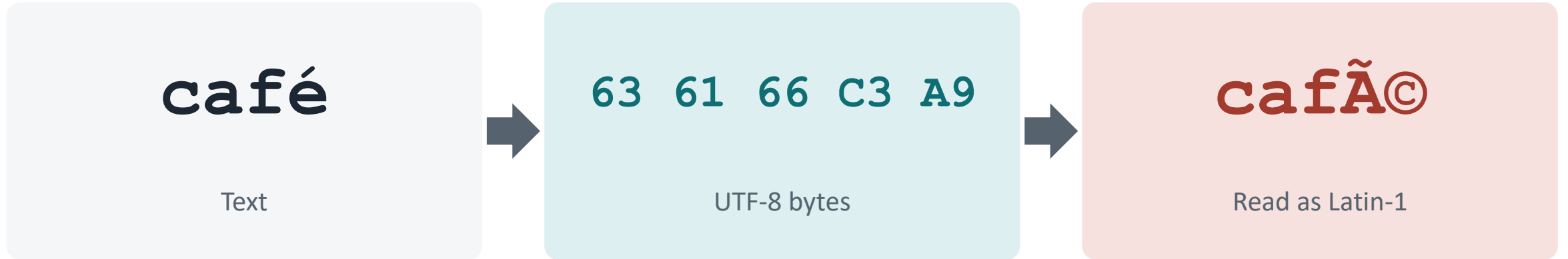
time

separator

UTC Time Format

Largest unit first, so it sorts correctly as text. Always keep the time zone.

Unicode and UTF-8: bytes need a decoder



```
>>> "café".encode("utf-8").decode("latin-1")  
'cafÃ©'
```

Know the encoding of every file you read.

ISO 3166 and ISO 4217: codes are join keys

- ISO 3166 defines internationally recognised codes for country names, while ISO 4217 builds on it to define standard codes for global currencies

GB United Kingdom

DE Germany

BR Brazil

NA Namibia

GBP

EUR

BRL

ISO 4217 currency codes

```
>>> pd.read_csv("countries.csv")
   code      name
0    GB  United Kingdom
1    DE    Germany
2    BR    Brazil
3   NaN  Namibia

>>> pd.read_csv("countries.csv",
...              keep_default_na=False)
```

**Many data failures happened because a tool
guessed a type.**

Standards exist so that nobody has to guess.

Higher-level data structures

Table

Records (rows) and fields (columns)

Weeks 3, 5

Key

A field that identifies a record and links tables

Week 5

Key-value pair

"ward": "B3"

Week 2

Nested document

Records inside records

Weeks 2, 5

Array

An ordered run of values of one type

Week 3

Graph

Nodes connected by edges

Week 11

Tidy data

Years spread across columns

borough	2023	2024	2025
Camden	412	398	405
Hackney	377	391	402



Tidy

borough	year	count
Camden	2023	412
Camden	2024	398
Camden	2025	405
Hackney	2023	377
...	...	...

Each variable is a column

Each observation is a row

Each type of unit is a table

Before you use any dataset, ask three questions

1

Where did it come from?

Provenance

Who made it, when, how, and what has changed since

2

What may I do with it?

Licensing

Publicly visible is not the same as free to reuse

3

Is anyone identifiable?

UK GDPR and ethics

Personal data needs a lawful basis; research may need ethics review

GITHUB: REPRODUCIBLE WORKFLOWS

Reproducibility is a professional standard

Same data + same code + same environment = same result, for anyone

You, in six months

Cannot remember which cleaning step dropped 400 rows, or why.

Your collaborator

Cannot run your notebook: different paths, packages, data.

Your reader or marker

Cannot check the claim, so cannot trust it.

Undocumented analysis is not a finished analysis.

What version control gives you

A full history

Every change, with who made it, when, and why

A way back

Return to any earlier version of any file

Parallel work

Try an idea on a branch without breaking what works

A shared copy

Collaborators work on the same project without emailing files

- **Git** is the tool on your laptop. **GitHub** is a website that hosts Git repositories and adds collaboration.

The Git model: three places a change can be

Working directory

Your files as they are on disk now

`git add`



Staging area

Changes chosen for the next commit

`git commit`



Repository

The history of commits

```
git status      # where is each change?
```

Run it constantly.

An example: your first commit

```
git clone https://github.com/<you>/st445-week1-demo.git
cd st445-week1-demo
git status          # working tree clean

# edit README.md, save
git status          # modified: README.md (not staged)

git add README.md
git status          # changes to be committed: README.md

git commit -m "Add data source and licence to README"
git log --oneline
```

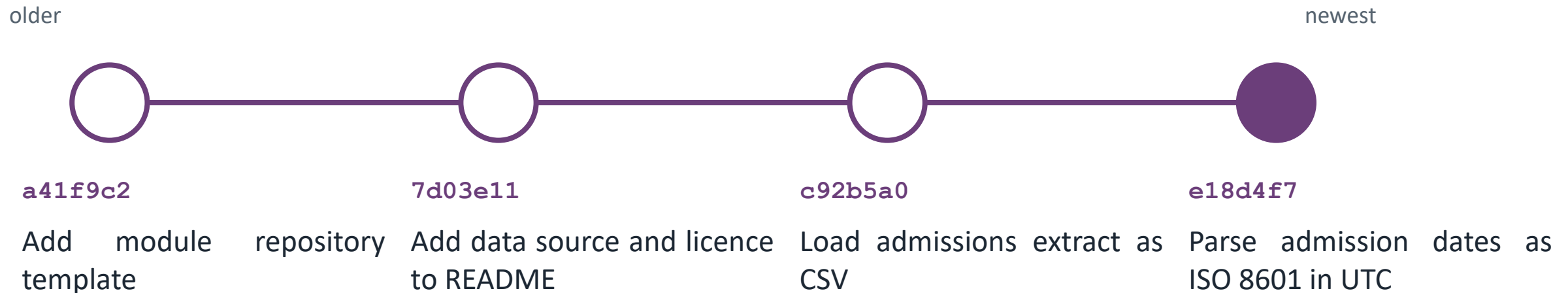
Watch the change move

Working directory

Staging area

Repository

A commit is a snapshot with a message



Each dot records every file: who, when, and why. You can return to any of them.

A useful commit message

Tells you nothing

update

fix

stuff

asdfgh

final version

Tells you what and why

Parse admission dates as ISO 8601 in UTC

Drop 14 duplicate patient rows before join

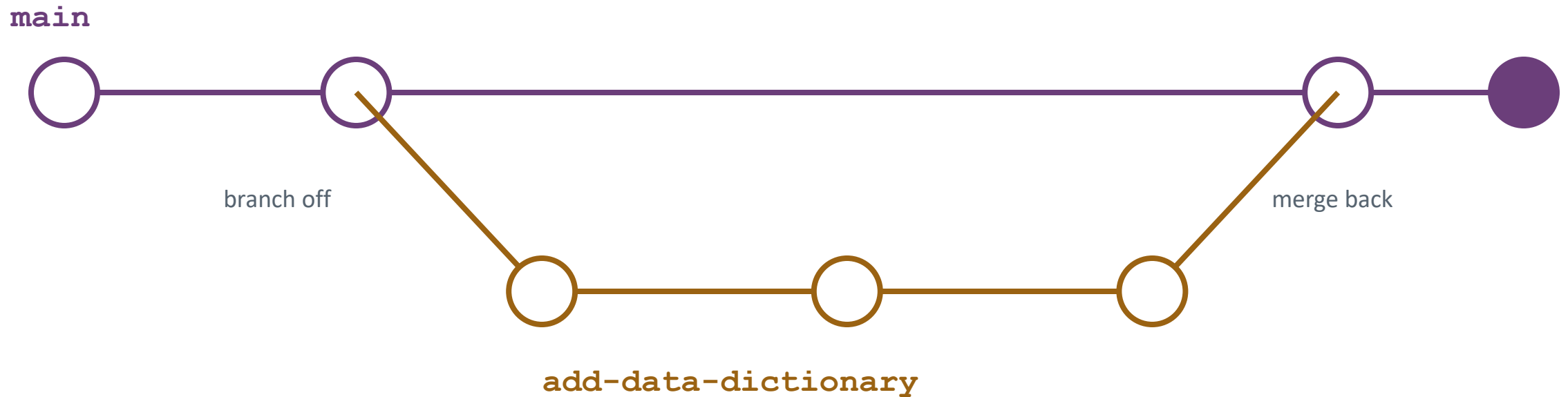
Read country codes with keep_default_na=False

Add data dictionary for admissions extract

Move raw data to data/raw and make read-only

- Short summary in the imperative: what changed and why, if not obvious. One change per commit.

Branches: try something without breaking main



```
git switch -c add-data-dictionary    # create the branch and move onto it
```

Your laptop and GitHub

Your laptop

Local repository: working directory, staging area and full history

`git clone`

copy the remote once

`git push`

send your commits

`git pull`

fetch + merge theirs

GitHub

Remote repository, called origin.
What your partner sees.

- Nothing reaches GitHub until you push.

GitHub for collaboration

README

The front door: what this is, how to run it, where the data came from

`.gitignore`

Files Git should never track

Issues

A to-do list attached to the code

Pull requests

A proposal to merge a branch, open for discussion

Code review

Comments on specific lines before anything merges

The pull request workflow



```
git push -u origin add-data-dictionary  
# then on GitHub: Compare & pull request
```

Be specific about the code and suggest a fix.

A useful review comment

"Line 12 reads dates without a time zone. Could we parse them as UTC so that this matches the API data?"

What never goes into the repository

```
# .gitignore

# secrets
.env

# large or private data
data/raw/
*.parquet

# generated files
__pycache__/
.ipynb_checkpoints/
.venv/
.DS_Store
```

Secrets

A pushed API key should be treated as compromised, even after you delete it.

Large or private data

Keep it out; the README says how to get it.

Generated files

Rebuilt by running the code, so never stored.

A project a stranger can run

```
st445-project/  
├── README.md  
├── .gitignore  
├── pyproject.toml    uv.lock  
│   (or environment.yml)  
├── data/  
│   ├── raw/         never edited  
│   └── processed/    rebuilt by code  
├── notebooks/  
├── src/  
└── outputs/figures/
```

Capture the environment

uv

```
uv add pandas pyarrow  
uv sync                # rebuild it anywhere
```

conda

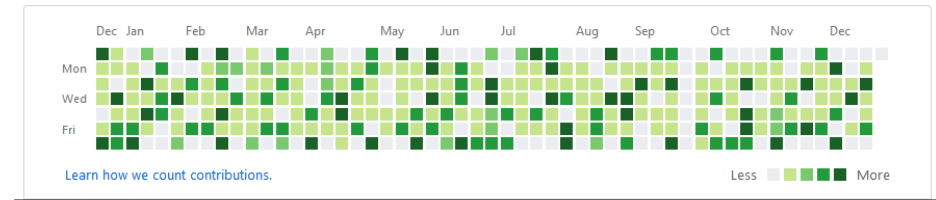
```
conda env export --from-history \  
> environment.yml
```

Same code, different package versions, different results.

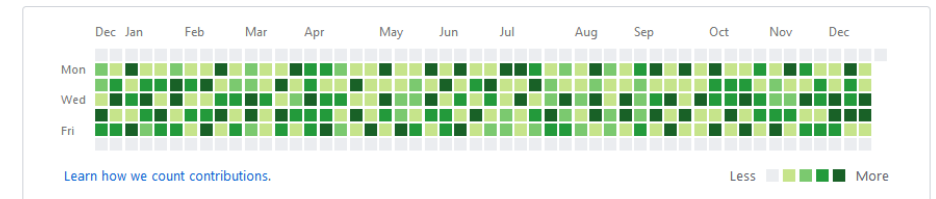
Resources for using git/GitHub

- An [easy git Cheatsheet](#), by Nina Jaeschke and Roger Dudler
- [git - the simple guide](#) by Roger Dudler
- Free stuff for students! <https://education.github.com/pack>

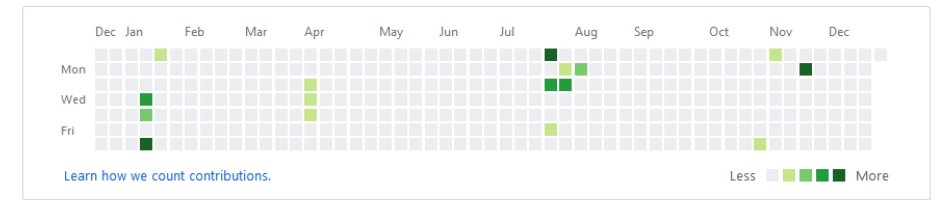
"The Open Source Enthusiast"



"My company uses github"



"Let's learn programming. For real this time"



"The curious"





Summary



We introduce the concepts of Data.



We discussed different data structures.



We introduced GitHub.